

Lista Avaliativa 3 - Respostas

1)

A exclusão mútua pode ser obtida tanto por TSL quanto pela desabilitação de interrupções, mas ambas são consideradas soluções primitivas com desvantagens significativas.

TSL (Test and Set Lock): A TSL é uma instrução de hardware que funciona de forma atômica (indivisível). Ela lê o valor de uma variável de trava (*lock*) na memória, armazena este valor em um registrador e, simultaneamente, escreve um valor "1" (indicando "travado") de volta na mesma posição de memória.

Para obter exclusão mútua, um processo, antes de entrar em sua região crítica, executa a TSL em um *loop*. Ele verifica o valor retornado pela instrução:

- Se o valor retornado for "0" (destravado), o processo obteve a trava com sucesso (pois a TSL a alterou para "1") e pode entrar na região crítica.
- Se o valor retornado for "1" (travado), o processo continua no *loop*, testando a trava repetidamente até que ela seja liberada. Ao sair da região crítica, o processo simplesmente escreve "0" de volta na variável de trava.

Desabilitação de Interrupções: Em um sistema com um único processador, a exclusão mútua pode ser alcançada desabilitando todas as interrupções de hardware imediatamente antes de entrar na região crítica e reabilitando-as ao sair. Como as trocas de contexto são geralmente disparadas por interrupções (como a do temporizador), desabilitá-las garante que o processo atual não será preemptado, podendo assim acessar recursos compartilhados sem o risco de interferência de outros processos.

Avaliação e Desvantagens: Nenhuma das duas é uma boa solução para uso geral em aplicações modernas:

- **Desvantagens da TSL:**
 - **Espera Ocupada (*Busy-Waiting*):** A principal desvantagem é que o processo consome tempo de CPU em um *loop* inútil enquanto espera pela trava. Isso desperdiça recursos e é ineficiente, especialmente se a espera for longa.
 - **Inversão de Prioridade:** Um processo de alta prioridade pode ficar preso em espera ocupada por uma trava mantida por um processo de baixa prioridade, que por sua vez pode nunca ser escalonado para liberar a trava.
- **Desvantagens da Desabilitação de Interrupções:**
 - **Perigoso para o Usuário:** Conceder a um processo de usuário o poder de desabilitar interrupções é extremamente arriscado. Um programa malicioso ou com erro poderia travar todo o sistema ao não reabilitar as interrupções. Por isso, essa técnica só é segura para uso dentro do próprio núcleo do sistema operacional.
 - **Ineficaz em Sistemas Multiprocessadores:** Desabilitar as interrupções em uma CPU não impede que outra CPU acesse a mesma memória concorrentemente, tornando a solução inútil em hardware moderno.
 - **Aumento da Latência:** Se as interrupções permanecerem desabilitadas por muito tempo, o sistema pode deixar de responder a eventos importantes, como a chegada de dados da rede.

ou do disco.

2)

Mutexes (Mutual Exclusion Locks): Um *mutex* é uma primitiva de sincronização de mais alto nível que protege o acesso a uma região crítica. Diferente da espera ocupada da TSL, quando um *thread* tenta adquirir um *mutex* que já está travado, o sistema operacional o coloca em estado de bloqueio. O *thread* é movido para uma fila de espera e não consome CPU. Quando o *thread* que detém o *mutex* o libera, o sistema operacional acorda um dos *threads* que estavam na fila, permitindo que ele adquira a trava e prossiga.

Implementação em Pthreads: A biblioteca Pthreads oferece uma implementação padrão para *mutexes* com as seguintes funções principais:

- `pthread_mutex_init()`: Inicializa uma variável do tipo *mutex*.
- `pthread_mutex_lock()`: Tenta adquirir a trava. Se o *mutex* estiver livre, o *thread* o adquire e continua. Se estiver ocupado, o *thread* é bloqueado até que o *mutex* seja liberado.
- `pthread_mutex_unlock()`: Libera o *mutex*, permitindo que outro *thread* que esteja esperando possa adquiri-lo.
- `pthread_mutex_destroy()`: Destrói o *mutex* após o uso.

Variáveis de Condição: *Mutexes* são excelentes para garantir a exclusão mútua, mas são insuficientes quando um *thread* precisa esperar por uma condição específica para poder continuar seu trabalho (ex: um consumidor esperando que um buffer deixe de estar vazio). Um *thread* não pode simplesmente manter o *mutex* travado e esperar em um *loop*, pois isso impediria que outros *threads* (como o produtor) alterassem a condição.

As variáveis de condição resolvem esse problema, trabalhando em conjunto com os *mutexes*. Elas permitem que um *thread*:

1. Trave o *mutex* para verificar uma condição de forma segura.
2. Se a condição não for satisfeita, ele chama `pthread_cond_wait()`. Essa função, de forma atômica, libera o *mutex* e coloca o *thread* para dormir.
3. Outro *thread* (ex: o produtor) pode então adquirir o *mutex*, alterar a condição (ex: adicionar um item ao buffer) e chamar `pthread_cond_signal()` ou `pthread_cond_broadcast()` para acordar um ou todos os *threads* que estavam esperando naquela variável de condição.
4. O *thread* acordado, ao retornar de `pthread_cond_wait()`, automaticamente readquire o *mutex* e pode reavaliar a condição para continuar sua execução.

3)

O Round Robin é um dos algoritmos de escalonamento mais simples e justos, projetado especialmente para sistemas de tempo compartilhado. Ele é um algoritmo preemptivo. Seu funcionamento baseia-se nos seguintes passos:

1. O sistema operacional mantém uma fila circular de todos os processos no estado "pronto".
2. A cada processo é alocada uma pequena fatia de tempo fixa, chamada de *quantum*.
3. O escalonador seleciona o primeiro processo da fila e o despacha para a CPU.
4. O processo executa até que uma de duas coisas aconteça:

- Ele termina ou bloqueia (ex: aguardando uma operação de E/S) antes que o *quantum* expire. Neste caso, o escalonador imediatamente seleciona o próximo processo da fila.
- O *quantum* de tempo expira. O sistema operacional é notificado por uma interrupção de temporizador, preempta (interrompe forçadamente) o processo, move-o para o final da fila de "prontos" e despacha o próximo processo da fila.

Exemplo: Considere três processos (P1, P2, P3) que chegam ao mesmo tempo com os seguintes tempos de execução (em milissegundos):

- P1: 20 ms
- P2: 5 ms
- P3: 8 ms

Vamos assumir um quantum de 5 ms. A execução ocorreria da seguinte forma:

Processo	P1	P2	P3	P1	P3	P1	P1
Tempo (ms)	0-5	5-10	10-15	15-20	20-23	23-28	28-33

- **0-5 ms:** P1 executa por 5 ms e é preemptado (ainda restam 15 ms). A fila de prontos agora é [P2, P3, P1].
- **5-10 ms:** P2 executa por 5 ms e termina. A fila de prontos é [P3, P1].
- **10-15 ms:** P3 executa por 5 ms e é preemptado (ainda restam 3 ms). A fila é [P1, P3].
- **15-20 ms:** P1 executa por mais 5 ms (agora restam 10 ms). A fila é [P3, P1].
- **20-23 ms:** P3 executa por 3 ms e termina. A fila é [P1].
- **23-33 ms:** P1 executa em dois *quanta* seguidos (e mais um parcial) até terminar.

A grande vantagem do Round Robin é que ele oferece tempos de resposta razoáveis para processos interativos, pois nenhum processo precisa esperar mais do que $(n-1) * \text{quantum}$ para receber sua próxima fatia de tempo.

4)

Este algoritmo de escalonamento prioriza os processos limitados por E/S ou processos interativos.

A lógica é a seguinte:

- Um processo limitado por E/S executa por um curto período de tempo antes de bloquear para realizar uma operação de E/S (como ler um dado do disco ou da rede). Portanto, ele utiliza apenas uma pequena fração f do seu *quantum*. Como f é pequeno, o valor de $1/f$ será grande, resultando em uma alta prioridade.
- Um processo limitado por CPU tende a usar todo o seu *quantum* de tempo em cálculos. Neste caso, f será próximo de 1, e o valor de $1/f$ será baixo, resultando em uma baixa prioridade.

Vantagem de Priorizar Processos Limitados por E/S: A principal vantagem de dar alta prioridade a esses processos é a melhora da responsividade e da utilização geral do sistema.

1. **Melhor Responsividade:** Processos interativos (como editores de texto, navegadores web) são tipicamente limitados por E/S, pois passam a maior parte do tempo esperando por uma entrada do usuário (teclado, mouse). Ao dar-lhes alta prioridade, o sistema garante que eles sejam executados

imediatamente assim que recebem uma entrada, proporcionando uma experiência de usuário fluida e sem atrasos perceptíveis.

2. **Maior Utilização dos Recursos:** Um processo limitado por E/S precisa de um curto surto de CPU para processar dados e rapidamente iniciar a próxima operação de E/S. Ao escaloná-lo rapidamente, o sistema permite que ele mantenha os dispositivos de E/S (que são lentos) ocupados. Enquanto o dispositivo de E/S está trabalhando, a CPU pode ser alocada a outros processos (como os *CPU-bound*), maximizando o paralelismo entre a CPU e os dispositivos de E/S e, conseqüentemente, a vazão (*throughput*) do sistema.

5)

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

int contador = 0;

// Variáveis para a Solução de Peterson
int flag[2] = {0, 0};
int turn = 0;

void *thread_sem_controle(void *arg)
{
    int id = *(int *)arg;

    for (int i = 0; i < 5; i++)
    {
        // SEÇÃO CRÍTICA
        int valor_local = contador;
        printf("Thread %d: leu contador = %d\n", id, valor_local);

        sched_yield();

        contador = valor_local + 1;
        printf("Thread %d: incrementou contador para %d\n", id, contador);
        // FIM DA SEÇÃO CRÍTICA

        sleep(1);
    }

    return NULL;
}

void *thread_peterson(void *arg)
{
    int id = *(int *)arg;
    int outro = 1 - id;

    for (int i = 0; i < 5; i++)
    {
```

```

    // ENTRADA NA SEÇÃO CRÍTICA
    flag[id] = 1;
    turn = outro;

    while (flag[outro] == 1 && turn == outro)
    {
        // Espera ocupada
    }

    // SEÇÃO CRÍTICA
    int valor_local = contador;
    printf("Thread %d: leu contador = %d\n", id, valor_local);

    sched_yield();

    contador = valor_local + 1;
    printf("Thread %d: incrementou contador para %d\n", id, contador);
    // FIM DA SEÇÃO CRÍTICA

    flag[id] = 0;

    sleep(1);
}

return NULL;
}

int main()
{
    pthread_t threads[2];
    int ids[2] = {0, 1};

    printf("=====\n");
    printf("VERSÃO 1: SEM CONTROLE DE CONCORRÊNCIA\n");
    printf("=====\n");

    contador = 0;

    // Cria as threads sem controle
    for (int i = 0; i < 2; i++)
    {
        pthread_create(&threads[i], NULL, thread_sem_controle, &ids[i]);
    }

    // Aguarda finalização
    for (int i = 0; i < 2; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("\n>>> Valor final do contador (SEM controle): %d\n", contador);
    printf(">>> Valor esperado: 10\n\n");

    sleep(2);
}

```

```

printf("=====\n");
printf("VERSÃO 2: COM SOLUÇÃO DE PETERSON\n");
printf("=====\n");

// Reset das variáveis
contador = 0;
flag[0] = 0;
flag[1] = 0;
turn = 0;

// Cria as threads com Peterson
for (int i = 0; i < 2; i++)
{
    pthread_create(&threads[i], NULL, thread_peterson, &ids[i]);
}

// Aguarda finalização
for (int i = 0; i < 2; i++)
{
    pthread_join(threads[i], NULL);
}

printf("\n>>> Valor final do contador (COM Peterson): %d\n", contador);
printf(">>> Valor esperado: 10\n\n");

return 0;
}

```

6)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <dirent.h>

#define MAX_CMD_LEN 1024
#define MAX_ARGS 64

void list_dir() {
    DIR *d;
    struct dirent *dir;
    d = opendir(".");

    if (d) {
        while ((dir = readdir(d)) != NULL) {
            // Não exibe os diretórios "." e ".."

```

```

        if (strcmp(dir->d_name, ".") != 0 && strcmp(dir->d_name, "..")
!= 0) {
            printf("%s\n", dir->d_name);
        }
    }
    closedir(d); // Fecha o diretório
} else {
    perror("Erro ao abrir diretório");
}
}

int main() {
    char cmd[MAX_CMD_LEN];
    char *args[MAX_ARGS];
    pid_t pid;

    while (1) {
        printf("mini-shell> ");
        fflush(stdout);

        if (fgets(cmd, sizeof(cmd), stdin) == NULL) {
            break;
        }

        cmd[strcspn(cmd, "\n")] = 0;

        // Analisa a linha de comando para separar o comando e seus
argumentos
        char *token;
        int i = 0;
        token = strtok(cmd, " "); // Divide a string por espaços
        while (token != NULL) {
            args[i] = token;
            i++;
            token = strtok(NULL, " ");
        }
        args[i] = NULL; // O último elemento do array de argumentos deve
ser NULL para o execve

        // Verifica se algum comando foi digitado
        if (args[0] == NULL) {
            continue; // Se o usuário apenas apertar Enter, volta ao início
do loop
        }

        // Verificar se é um comando interno
        if (strcmp(args[0], "exit") == 0) {
            break; // Comando para sair do shell
        }

        if (strcmp(args[0], "ls") == 0) {
            list_dir();
            continue;
        }
    }
}

```

```

    }

    // Cria um novo processo para executar o comando externo
    pid = fork();

    if (pid < 0) {
        perror("Erro no fork");
        exit(1);
    } else if (pid == 0) {
        if (execvp(args[0], args) < 0) {
            perror("Comando não encontrado");
            exit(1);
        }
    } else {
        wait(NULL);
    }
}

printf("\nSaindo do mini-shell.\n");
return 0;
}

```

7)

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <semaphore.h>
#include <string.h>

#define SEM_GARFO_NOME_BASE "/garfo_filosofo"

void ciclo_do_filosofo(int id, int num_filosofos)
{
    // Identifica os garfos à esquerda e à direita do filósofo
    int garfo_esquerda = id;
    int garfo_direita = (id + 1) % num_filosofos;

    // Gera os nomes dos semáforos para os garfos
    char nome_sem_esquerda[50], nome_sem_direita[50];
    sprintf(nome_sem_esquerda, "%s%d", SEM_GARFO_NOME_BASE,
garfo_esquerda);
    sprintf(nome_sem_direita, "%s%d", SEM_GARFO_NOME_BASE, garfo_direita);

    // Abre os semáforos existentes (criados pelo processo pai)
    sem_t *sem_garfo_esquerda = sem_open(nome_sem_esquerda, 0);
    sem_t *sem_garfo_direita = sem_open(nome_sem_direita, 0);
}

```

```

    if (sem_garfo_esquerda == SEM_FAILED || sem_garfo_direita ==
SEM_FAILED)
    {
        perror("Filósofo não conseguiu abrir os semáforos");
        exit(1);
    }

    while (1)
    {
        // PENSAR
        printf("Filósofo %d está PENSANDO.\n", id);
        sleep(rand() % 3 + 1); // Pensa por um tempo aleatório

        printf("Filósofo %d está com FOME e vai tentar pegar os garfos.\n",
id);

        // PEGAR GARFOS (Entrada na Seção Crítica)
        // ESTRATÉGIA PARA EVITAR DEADLOCK:
        // O último filósofo pega o garfo da direita primeiro, enquanto os
outros
        // pegam o da esquerda primeiro. Isso quebra a dependência
circular.
        if (id == num_filosofos - 1)
        {
            sem_wait(sem_garfo_direita);
            printf("Filósofo %d pegou o garfo da DIREITA (%d).\n", id,
garfo_direita);
            sem_wait(sem_garfo_esquerda);
            printf("Filósofo %d pegou o garfo da ESQUERDA (%d).\n", id,
garfo_esquerda);
        }
        else
        {
            sem_wait(sem_garfo_esquerda);
            printf("Filósofo %d pegou o garfo da ESQUERDA (%d).\n", id,
garfo_esquerda);
            sem_wait(sem_garfo_direita);
            printf("Filósofo %d pegou o garfo da DIREITA (%d).\n", id,
garfo_direita);
        }

        // COMER
        printf(">>> Filósofo %d está COMENDO. <<<\n", id);
        sleep(rand() % 3 + 1); // Come por um tempo aleatório

        // DEVOLVER GARFOS (Saída da Seção Crítica)
        printf("Filósofo %d TERMINOU de comer e vai devolver os garfos.\n",
id);
        sem_post(sem_garfo_esquerda); // Libera o garfo da esquerda
        sem_post(sem_garfo_direita); // Libera o garfo da direita
    }

    // Este código nunca será alcançado devido ao while(1),
sem_close(sem_garfo_esquerda);

```

```

    sem_close(sem_garfo_direita);
}

int main(int argc, char *argv[])
{
    if (argc != 2)
    {
        fprintf(stderr, "Uso: %s <numero_de_filosofos>\n", argv[0]);
        return 1;
    }

    int num_filosofos = atoi(argv[1]);

    if (num_filosofos < 2)
    {
        fprintf(stderr, "O número de filósofos deve ser pelo menos 2.\n");
        return 1;
    }

    // Criação e Inicialização dos Semáforos (Garfos)
    // Cada garfo é um semáforo.
    for (int i = 0; i < num_filosofos; i++)
    {
        char nome_sem[50];
        sprintf(nome_sem, "%s%d", SEM_GARFO_NOME_BASE, i);

        sem_t *sem = sem_open(nome_sem, O_CREAT, 0644, 1);
        if (sem == SEM_FAILED)
        {
            perror("Erro ao criar semáforo no processo pai");
            exit(1);
        }
        sem_close(sem);
    }

    // Criação dos Processos Filósofos
    pid_t pids[num_filosofos];
    for (int i = 0; i < num_filosofos; i++)
    {
        pids[i] = fork();

        if (pids[i] < 0)
        {
            perror("Erro no fork");
            exit(1);
        }

        if (pids[i] == 0)
        {
            ciclo_do_filosofo(i, num_filosofos);
            exit(0);
        }
    }
}

```

```
    printf("Pai esperando todos os filósofos terminarem (Ctrl+C para
encerrar)\n");
    for (int i = 0; i < num_filosofos; i++)
    {
        wait(NULL);
    }

    printf("Pai limpando os semáforos...\n");
    for (int i = 0; i < num_filosofos; i++)
    {
        char nome_sem[50];
        sprintf(nome_sem, "%s%d", SEM_GARFO_NOME_BASE, i);
        if (sem_unlink(nome_sem) == -1)
        {
            perror("Erro ao desvincular semáforo");
        }
    }

    printf("Programa finalizado.\n");
    return 0;
}
```